SAMS Kernel

Developing with a real TI: running, debugging, editing

Version 1.2.0 · 28 October 2026
For a TI-99/4A with an F18A video card and a SAMS memory card
MIT License, © 2026 Rebecca Sutter

Contents

1. What you need on the TI
2. Level A: running on the TI, and the first-run ladder
3. Debugging on a TI that has no debugger
4. Level B: editing on the TI
5. Level C: everything on the TI (not possible today)
6. Differences between the TI and the simulator, to expect

Can you develop for the kernel *on* a TI? Partly, and this document says exactly how far, what is checked and what is not. The kernel's build is a PC program (Python and the xdt99 assembler), and the kernel itself loads from a cartridge into SAMS; both facts shape what a TI can do. Three levels:

Level	What you do on the TI	Works today?
A. The TI as the test machine	run what the PC built, look at it, test it	yes (with the caveats below); the normal way
B. The TI as the editor too	write the source on the TI, move the files to the PC to build	yes; the sources are kept safe for it (part 4)
C. Everything on the TI	edit, assemble, build and run with no PC	**no**: part 5 says what it would take

What is verified. All of this project is verified in a simulator (a TMS9900, the F18A, SAMS, a keyboard, TI's own console ROM and disk controller ROM), not on a real TI-99/4A. No image from this project has yet been run on a real FinalGROM 99 or a real F18A and SAMS card. Treat the first run on real hardware as a test, and use the ladder in part 2.

1. What you need on the TI

A TI-99/4A with an **F18A** video card (or a PICO9918), a **SAMS** card of at least what the application asks for (512K by default), and a way to load a cartridge image: a **FinalGROM 99** or any cartridge that presents 8K banks selected by writes to `>6000 + 2n` (the images are in the "8" format: the build's output is named `NAME8.bin`). The F18A and SAMS are not optional: the loader refuses to start without them, and says so on the screen (part 2).

2. Level A: running on the TI, and the first-run ladder

Build on the PC (`python3 tools/skbuild.py APPDIR`), copy `build/APPDIR/NAME8.bin` to the cartridge's storage, and select it. For anything you will rely on, **climb the ladder**: each rung tells you something the one below cannot.

1. **The simulator and js99er.** The tests and a browser emulator (F18A and SAMS on) find nearly every programming mistake. They are not the machine: they do not model timing, and a real disk controller behaves differently from a stand-in in ways that matter (the file service found two: after the read that reports end of file TI's controller has already closed the file, and a missing file opens with error 2).
2. **`examples/hello`** (`images/hello8.bin`). Power on, select it, type: the screen is blue, a title bar, your typing echoed. If this works, the loader, SAMS, the F18A, the keyboard and the kernel all work.
3. **`examples/diag`** (`images/skdiag8.bin`). The key decoder (every chord shows its code and modifiers), the whole glyph set, the colour roles; **CTRL T** runs the kernel's self-test: hardware, paging, page pool, a module call, the keyboard, the screen routines, DMA, and a write, read and delete on DSK1 (it needs a disk in drive 1; a failing check shows its step and the DSR's error: step x 100 + error).
4. **`examples/modes`** (video), **`examples/music`** (sound: listen for the tune and the A), **`examples/browser`** (a catalog and a text file from your own disk).
5. **Your application.**

What the loader says when it refuses. It shows one of these in plain text and waits for FCTN = to return to the title:

Message	What it means
NEEDS A SAMS MEMORY CARD WITH 512K OF MEMORY OR MORE TO RUN	no mapper answered at CRU `>1E00`, or the card has fewer pages than the application's `min_pages` (the size probe reads the pages it can tell apart)
THE MEMORY CARD FAILED ITS TEST AT PAGE >xx	a page from `>10` up read back what it was not given: the page number is the first that failed, so a 4K area of RAM (a chip or its socket) is bad
NEEDS AN F18A VIDEO CARD FOR ITS 80-COLUMN SCREEN AND COLOR	the F18A did not identify itself: not fitted, not connected, or its firmware does not answer the status register the loader reads

If none of them appears and the screen stays dark, the loader got as far as starting the kernel; suspect the application (below).

3. Debugging on a TI that has no debugger

There is no monitor or single-stepper. What you have, and it is enough for most things:

- **`kernel/dbg.a99`: a routine that shows registers.** `COPY "dbg.a99"` in your module, then `BL @DBG` anywhere: the bottom row shows R0-R3 in hex and waits for a key; every register is kept, so you can put it between any two instructions. Load a register with what you want to see first (`MOV @SEL,R0`). It is tested in the simulator (`tests/test_kernel.py dbg`).
- **A heartbeat.** To see that the main loop is alive, show the frame counter in a corner from your `TICK`: `LI R0,COLS-5 / MOV @FRAME,R1 / LI R2,ABAR / BL @HEX4`. If it stops counting, your `TICK` (or something it calls) never returned: a loop with no exit, a stack overflow, or a call that lost its return address.
- **The self-test (CTRL T in `diag`)** after anything that touches paging, the stack or files.
- **Breadcrumbs.** Write a letter to a fixed cell as you pass milestones (`PUTC`); the last letter on a hung screen is where it stopped.
- **FCTN = quits** only if your `TICK` is running: a hung program needs the console's reset switch or power-off.

Typical symptoms. A blank screen after the refusal screens are gone: the module crashed in `INIT` (check you assembled to `AORG >C000` and kept the entry table first). A screen that stops updating but keys still beep: `TICK` runs too long (the frame counter). Garbled text colours in a mode with enhanced colour: you passed role attributes where flags belong (VIDEO-MODES.md part 5). A crash after a file call: you kept a value in `R6-R9` across it. Music stutters: your code runs long between `KIDLE` calls. VIDEO-MODES.md, PROGRAMMERS-GUIDE.md part 14 and CONVENTIONS.md have the rest.

4. Level B: editing on the TI

A TI text file is **DIS/VAR 80**: lines of 80 columns of ASCII. Any editor on the TI that writes that kind of file (TI-Writer, the Editor/Assembler's editor, a word processor that saves plain text) can edit the sources. Two things make it safe:

- **Every source line is at most 80 columns, plain ASCII, no tabs.** An editor cuts or splits a longer line; the shipped sources had 99 lines that would have been damaged (a string or a data list cut short is a silent bug). `tools/lint80.py` checks the rule, `tools/lint80.py --fix` repairs a file without changing a byte of the program (it moves a long comment above its line, re-wraps a long whole-line comment, splits a long string or data list; the build's images come out identical), and the test group `sources` checks every source in the project. Keep your own sources to it: run `lint80.py` before you copy them to the TI.
- **Moving files.** A disk image carries them both ways. xdt99's `xdm99` makes and reads TI disk images (tested here with `kernel/pool.a99`: out and back, the same text):

```
python3 xdm99.py -X sssd disk.dsk                        a blank single-sided single-density disk image
python3 xdm99.py disk.dsk -a main.a99 -n MAIN -f DIS/VAR80     put a source file on it, as the TI will see it: DSK1.MAIN
python3 xdm99.py disk.dsk                                its catalog
python3 xdm99.py disk.dsk -e MAIN -o main.a99            take a file back off
```

How the image gets to and from the real machine (a TIPI, a CF7+, a floppy written by a PC, a serial link) depends on your hardware; I have not tried any of them.

The loop is then: edit on the TI, move the files to the PC, `skbuild.py`, move the image to the cartridge, run. It works, and it is slow: do most of the iterating in the simulator and js99er, and use the TI to confirm.

5. Level C: everything on the TI (not possible today)

Three things are missing, and none of them is small. What I measured about the sources (23 files, 5,100 lines): they use the assembler's **macros** (`.enter`, `.leave`, `.push`, `.pop`, `.callmod`, `.intsvc`: 234 uses), **`STRI`** (102), **`COPY`** (36), **labels longer than six characters** (30), and **parenthesised expressions** (`(RTEXT*16+RBG)*256`).

1. **An assembler on the TI that accepts them.** I have not checked which TI-resident assemblers handle macros, long labels and `STRI`. The Editor/Assembler's assembler, as I know it, has no macros and short labels, so I expect it cannot assemble these sources unchanged: check before you build on it. (The alternatives are a macro-free flavour of the sources, which a script could generate, or a native assembler that takes the xas99 dialect.)
2. **A way to run without the cartridge.** The kernel and every module live in SAMS pages that the cartridge's loader copies in. The Editor/Assembler can load object code to an `AORG` address, and a module assembled at `>C000` would land in whatever page is mapped there (the identity map has `>0C-0D`, which is slot C's default pair), so a *single-module* program could in principle be loaded that way; modules in other pages cannot, because the loader does not map pages. A real answer is a small **disk loader**: a stage-0 program, run from the Editor/Assembler or from XBasic, that reads the kernel and module images from disk files into SAMS pages (the kernel's own file service, `files.a99`, is about a kilobyte) and starts the kernel. That would let a TI with a disk drive (or a TIPI) run builds with no FinalGROM. It would need a build step that writes the images as disk files.
3. **The build itself.** `skbuild.py` assigns pages, writes the application's equates, assembles the kernel and exports its symbols so that each module can `COPY "kernel.equ"`, packs the banks, and compiles music. On the TI all of that has to exist in a TI language. This is the big one.

None of that exists here; I have not built, tested or estimated it beyond the sketch above. If you want a part of it, the useful order is: the **disk loader** first (it removes the need for a cartridge on every test and is self-contained), then **macro-free generated sources** (so the Editor/Assembler can be tried), and only then a native build. Say which, and I will build that one and test it in the simulator against TI's real disk controller ROM.

6. Differences between the TI and the simulator, to expect

What	Where it is written down
a TI disk controller closes a file when a read reports the end, and answers 2 for a missing input file	PROGRAMMERS-GUIDE.md part 9
the console's frame counter advances by the byte at `>83FC`	HARDWARE.md
timing: the simulator runs a fixed number of instructions a frame; a real TI does not	the simulator's `VSYNC_INSNS`; test anything timing-sensitive on the TI
video: the renderer follows js99er's model of the F18A, not the F18A	VIDEO-MODES.md
sound: the player's bytes and timing are verified; how it sounds is not	MUSIC.md
a real card's SAMS, a real F18A's firmware and a PICO9918's quirks	nothing verified